

Data Validation Report – DEV-815

Migration Plan: MariaDB Galera to Percona XtraDB Cluster

Author: Thanusha Bai

Department: DevOps

Date: 11 September 2026

Why This Report Exists

When you migrate a database from one cluster to another, "it looks fine" is not good enough. You need proof, hard evidence that not a single row was lost and not a single value was changed. This report describes how I validated data integrity between a source database and a target database using two complementary techniques: row count validation and checksum validation. It also captures what I learned, where the limits are, and what a real production validation would need.

This report is written so that anyone, whether it is my future self, my lead, or another engineer picking this up later, can understand the approach and reproduce it.

The Core Problem With "It Migrated Fine"

A database migration can fail in two very different ways, and they require two very different detection methods.

The first failure mode is obvious: rows go missing. A table was not copied. A chunk of data was skipped. An entire database was forgotten. This is the loud, ugly failure, and it is easy to catch if you know to look for it.

The second failure mode is silent. Every row is there. The counts match perfectly. But something inside a row changed. A price became 500 instead of 5000. An email address picked up a stray character. A customer's name got mangled. Nothing is missing, but the data is wrong, and the row count will happily tell you everything is fine.

This is exactly why a serious validation plan needs two layers, not one.

The Two Validation Layers

Layer 1: Row Count Validation

What it is: Count the number of rows in a table on the source, count the same table on the target, and compare the two numbers.

What it catches:

- Entire tables that failed to migrate
- Missing rows from partial transfers
- Extra rows that should not exist on the target
- Off by thousands discrepancies from a broken copy job

What it misses:

- Any corruption inside a row, such as a changed value, a truncated field, or a swapped value
- Any data that is present but wrong

Speed: Very fast. A count query on a well indexed table runs in milliseconds. On a large table it may take seconds, but it is still cheap compared to a full scan.

Verdict: Row counts are your first pass sanity check. Run them immediately after migration. If they do not match, stop and investigate before going any further.

Layer 2: Checksum Validation

What it is: Instead of counting rows, you compute a fingerprint of the actual data in a table, a single hash value that represents the entire contents. You do this on the source, do it again on the target, and compare the two fingerprints.

How the fingerprint works, in plain terms:

1. Take every row and concatenate its values in a fixed order into one long string, such as 1|Alice|alice@example.com#2|Bob|bob@example.com
2. Run a hash function such as MD5, CRC32, or SHA1 over that string.
3. The output is a short, fixed length value like 54b4d623c9478fb3dc95e1e0e2f0156d.
4. Do the same on the target.
5. If the two hashes are identical, the data is byte for byte identical. If they differ, something, anything, is different, even if every row count matched.

What it catches:

- Silent corruption inside a row
- A single character changed in a single field

- A value swapped between rows
- Type conversion issues during migration
- Character set or collation problems that changed stored bytes

What it misses:

- Nothing, as long as the hashing is done consistently with the same order, separators, and encoding on both sides.

Speed: Slower than row counts, because you have to read every row, not just count them. On large tables this is a real cost, which is why in production you use a tool that chunks the work to avoid hammering the server.

Verdict: Checksums are your deep verification layer. Run them for all critical tables before declaring a migration successful.

The Practice Lab: What I Actually Did

To validate my understanding without access to production data, I built a small lab environment using Docker.

The setup:

- Two separate MySQL 8.0 containers, one acting as the source standing in for MariaDB Galera, and one as the target standing in for PXC.
- Both contain an identical sample database with a users table.
- I deliberately created a mismatch: the source had 2 rows, the target had 3, with an extra Charlie row on the target.

The validation sequence I ran:

1. Row count comparison, pulling the count from both sides.
 - Source: 2 rows
 - Target: 3 rows
 - Result: mismatch detected.
2. Checksum comparison, generating an MD5 hash of all rows on both sides.
 - Source hash: 0aad6d2122e8d39d2e0d9902b51997d2
 - Target hash: 54b4d623c9478fb3dc95e1e0e2f0156d
 - Result: mismatch confirmed, the data content differed, not just the count.
3. Correction, inserting the missing row into the source so both sides had the same 3 rows.
4. Re validation, running the checksum comparison again.

- Source hash: 54b4d623c9478fb3dc95e1e0e2f0156d
- Target hash: 54b4d623c9478fb3dc95e1e0e2f0156d
- Result: identical hashes, data confirmed consistent.

This is the same logical flow a real migration validation follows. The only difference is that in production you would run it across dozens or hundreds of tables, and you would use a tool to automate it.

Where This Leaves Room to Grow: The Real Tool

The manual MD5 approach I used in the lab teaches the concept, but it does not scale. In a real MariaDB to PXC migration, the industry standard tool is `pt-table-checksum`, part of Percona Toolkit.

What `pt-table-checksum` does differently:

- It splits each table into chunks based on the primary key, so it never locks the whole table at once.
- It writes checksum results into a `percona.checksums` table, so you can query and compare them later.
- It is cluster aware. With the recursion method set to cluster, it automatically discovers all Galera and PXC nodes.
- It respects load limits and replication lag, so it does not hurt production.

The key limitation to remember: `pt-table-checksum` requires every table to have a primary key or a unique key for chunking. Tables without one are skipped, which means those tables need manual validation, and ideally a primary key added before the migration happens.

How the output is interpreted:

Query the `percona.checksums` table and filter for rows where master count is not equal to this count, or master crc is not equal to this crc. Any rows returned by this query represent tables where source and target disagree. That is your mismatch list.

Design Decisions Behind This Validation Plan

These are the choices I made and the reasoning behind each one, so that anyone continuing this work understands not just what to do but why.

1. Row counts run first, across every table. This is the cheapest possible check and it eliminates whole categories of failure in one pass. If row counts do not match, there is no point running checksums yet, because we already know something is wrong.
2. Checksums run second, on all critical and business sensitive tables. Row counts cannot see inside a row, so this layer exists specifically to catch silent corruption. It is heavier, so it runs after the cheap check has already narrowed the field.
3. Tables without primary keys are flagged before migration, not after. `pt-table-checksum` cannot chunk a table without a unique key, so those tables get silently skipped. Finding them after cutover is too late. Finding them before means we can either add a key or plan a manual validation for them.
4. Every validation output is timestamped and saved. This turns validation from a one time action into an audit trail. If a question comes up two weeks later about whether a table was verified, the answer should be a file, not a memory.
5. The source cluster stays alive until the target passes cleanly. Validation that cannot be reversed is not validation, it is a gamble. Keeping the source available until the checks pass is what makes the cutover safe to attempt.
6. In production, this flow uses `pt-table-checksum` instead of manual MD5. The manual version proves the concept, but the tool is what makes it safe at scale, on live traffic, without locking tables.
7. A primary key is not optional in a real migration. It is the unit of chunking, the unit of comparison, and the unit of repair. Treating it as a prerequisite rather than a nice to have is what separates a plan that works in a lab from one that works in production.

The core principle underneath all of this is simple: a migration is not complete when the data has been copied. It is complete when the data has been proven to be identical.

What I Took Away From This

- Row counts and checksums are not competing methods. They answer two different questions. Row counts answer "is everything here?" Checksums answer "is everything correct?"
- A hash that matches is not a vague reassurance. It is a mathematical guarantee that the underlying bytes are identical.
- The real challenge in production is not the checksum logic. It is doing it safely, without locking tables, without flooding the

network, and without slowing the live application.

- pt-table-checksum exists precisely to solve that production problem, and its constraint around primary keys is the kind of detail that turns a "works in the lab" plan into a "works in production" plan.
- Documentation is not paperwork. It is what allows someone else to trust the result without having watched every command.

Appendix: Evidence and Screenshots

All commands were run on a Linux VM using two Docker MySQL 8.0 containers. Terminal output for every validation step was captured as evidence.

Screenshot 1 – Container setup

sudo docker ps

```
ThanushaBai@DevOps-Nomad-VM:~$ sudo docker ps
[sudo: authenticate] Password:
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
42a703f35587   grafana/loki:2.9.0  "/usr/bin/loki -conf..." 2 weeks ago    Up 3 minutes             loki
ThanushaBai@DevOps-Nomad-VM:~$ sudo docker run --name source-mysql -e MYSQL_ROOT_PASSWORD=my-secret-pw -p 3306:3306 -d mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
e3e5d1ac74c1: Pull complete
0d74d296605b: Pull complete
297d04cfe470: Pull complete
4c8a3e0d4e4b: Pull complete
a63160a5eda1: Pull complete
7534d1db9f8d: Pull complete
49ec2dab01d9: Pull complete
ab24264a27e9: Pull complete
edf85873f64e: Pull complete
6ef6c7b50a93: Pull complete
96d30d9fbee8: Pull complete
796812c73292: Download complete
af166387641d: Download complete
Digest: sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2b
Status: Downloaded newer image for mysql:8.0
7d85c7b028a6e8bddc9cd75cf30963aa828b067f9f7b9eedec32c9a274b0e06a
ThanushaBai@DevOps-Nomad-VM:~$ ^C
ThanushaBai@DevOps-Nomad-VM:~$ sudo docker run --name target-mysql -e MYSQL_ROOT_PASSWORD=my-secret-pw -p 3307:3306 -d mysql:8.0
08d628cdb54a50211b6cdb42c5c05de66b091f32e7eb8ff3bc9185b3b7fe9843
ThanushaBai@DevOps-Nomad-VM:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAME
5
08d628cdb54a   mysql:8.0      "docker-entrypoint.s..." 39 seconds ago    Up 37 seconds    33060/tcp, 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp    targ
et-mysql
7d85c7b028a6   mysql:8.0      "docker-entrypoint.s..." About a minute ago    Up About a minute    0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp    sour
ce-mysql
42a703f35587   grafana/loki:2.9.0  "/usr/bin/loki -conf..." 2 weeks ago      Up 6 minutes             loki
```

Screenshot 2 – Initial row count comparison (mismatch detected)

```
echo "SOURCE:" && sudo docker exec source-mysql mysql -uroot -pmy-secret-pw -N -e
"SELECT COUNT(*) FROM sample_db.users;" 2>/dev/null && echo "TARGET:" && sudo
docker exec target-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT COUNT(*) FROM
sample_db.users;" 2>/dev/null
```

```

ThanushaBai@DevOps-Nomad-VM:~$ echo "===== ROW COUNT VALIDATION ====="
echo ""
echo "SOURCE (port 3306):"
sudo docker exec source-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT COUNT(*) FROM sample_db.users;" 2>/dev/null
echo ""
echo "TARGET (port 3307):"
sudo docker exec target-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT COUNT(*) FROM sample_db.users;" 2>/dev/null
echo ""
echo "===== "
===== ROW COUNT VALIDATION =====

SOURCE (port 3306):
2

TARGET (port 3307):
3

=====

```

Screenshot 3 – Initial checksum comparison (mismatch confirmed)

```

echo "SOURCE:" && sudo docker exec source-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null && echo "TARGET:" && sudo docker exec target-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null

```

```

ThanushaBai@DevOps-Nomad-VM:~$ echo "===== CHECKSUM VALIDATION ====="
echo ""
echo "SOURCE (port 3306):"
sudo docker exec source-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null

echo ""
echo "TARGET (port 3307):"
sudo docker exec target-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null

echo ""
echo "===== "
===== CHECKSUM VALIDATION =====

SOURCE (port 3306):
0aad6d2122e8d39d2e0d9902b51997d2

TARGET (port 3307):
54b4d623c9478fb3dc95e1e0e2f0156d

=====

```

Screenshot 4 – Post correction checksum comparison (data consistent)

```
echo "SOURCE:" && sudo docker exec source-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null && echo "TARGET:" && sudo docker exec target-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null
```

```
ThanushaBai@DevOps-Nomad-VM:~$ echo "==== CHECKSUM VALIDATION (AFTER FIX) ====="
```

```
echo ""
```

```
echo "SOURCE (port 3306):"
```

```
sudo docker exec source-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null
```

```
echo ""
```

```
echo "TARGET (port 3307):"
```

```
sudo docker exec target-mysql mysql -uroot -pmy-secret-pw -N -e "SELECT MD5(GROUP_CONCAT(id, '|', name, '|', email ORDER BY id SEPARATOR '#')) FROM sample_db.users;" 2>/dev/null
```

```
echo ""
```

```
echo "=====
```

```
==== CHECKSUM VALIDATION (AFTER FIX) =====
```

```
SOURCE (port 3306):
```

```
54b4d623c9478fb3dc95e1e0e2f0156d
```

```
TARGET (port 3307):
```

```
54b4d623c9478fb3dc95e1e0e2f0156d
```

```
=====
```

End of Report